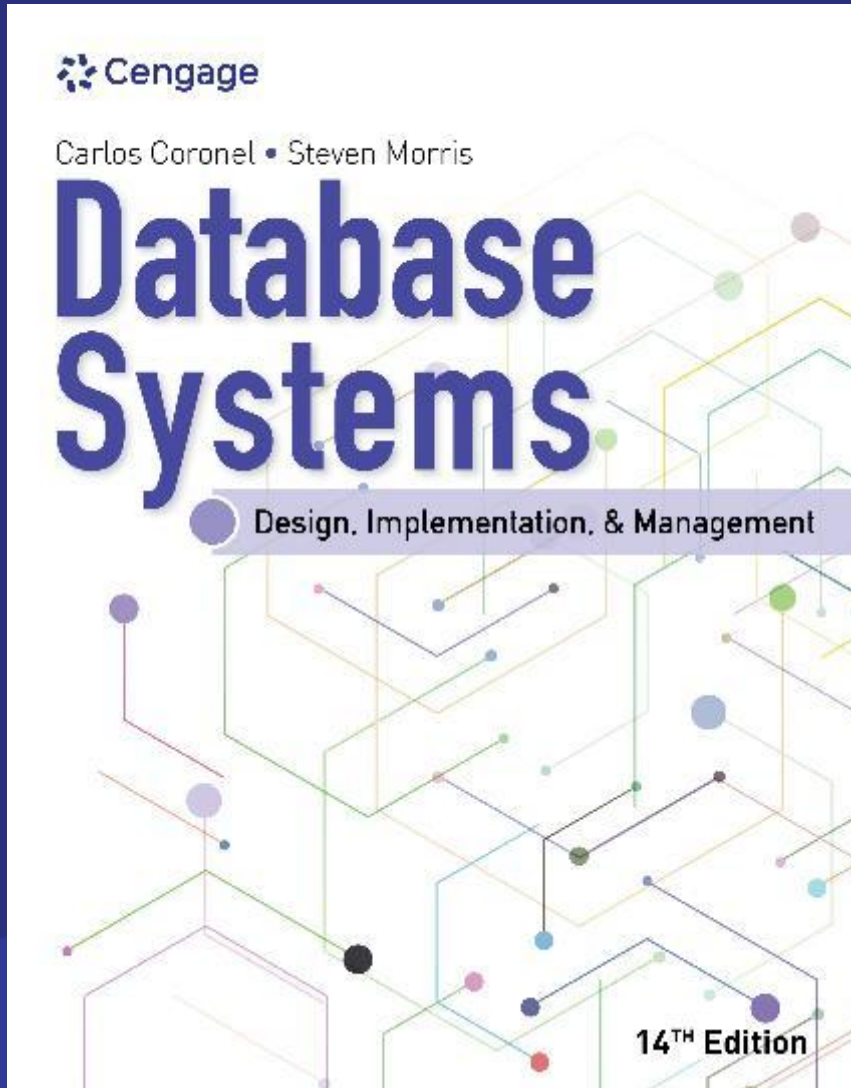




Database Systems: Design, Implementation, and Management, 14e

Module 6: Normalization of Database Tables

Contents

- Normalization and its role in the database design process
- Identify each of the normal forms: 1NF, 2NF, 3NF
- Explain how normal forms can be transformed from lower normal forms to higher normal forms
- Apply normalization rules to evaluate and correct table structures

Database Tables and Normalization (1 of 2)

- **Normalization** is the process of organizing database tables to reduce repeated data and improve data integrity.
 - Reduces data redundancy
 - Helps prevent data anomalies
 - Places attributes in the appropriate tables based on their dependencies
- Normalization is performed through a series of stages called **normal forms**. The three normal forms we will focus on are
 - **First Normal Form (1NF)**
 - **Second Normal Form (2NF)**
 - **Third Normal Form (3NF)**

Database Tables and Normalization (2 of 2)

- **Higher normal forms generally provide better data organization and reduce redundancy.**
 - However, additional normalization can make the database structure more complex.
 - For most business database applications, **Third Normal Form (3NF)** provides a good balance between:
 - Reducing data redundancy
 - Preventing data anomalies
 - Maintaining data integrity
 - Keeping the database practical and manageable
- **Note:** Higher normal forms such as BCNF, 4NF, and 5NF exist, but 3NF is sufficient for most business database designs.

The Need for Normalization

- **Normalization helps database designers create well-structured tables.**
- It is commonly used:
 - When designing a new database
 - When reviewing existing tables to improve their structure
- **The main goals of normalization are to:**
 - Reduce unnecessary data redundancy
 - Prevent **insert, update, and delete anomalies**
 - Improve data consistency and integrity
- Normalization uses **functional dependencies** to determine how attributes should be organized into tables.

The Normalization Process (1 of 4)

- The goal of normalization is to create **well-structured tables** with the following characteristics:
- Each table represents **one subject or entity**.
 - Each cell contains **one value only**—not a list or group of values.
 - Unnecessary **data duplication is minimized**.
 - Each non-key attribute depends on the **primary key**.
 - Tables are structured to reduce **insertion, update, and deletion anomalies**.

The Normalization Process (2 of 4)

- A common goal is to ensure that tables are normalized to at least **Third Normal Form (3NF)**.
- For most business databases, **3NF is usually sufficient**.
 - Higher normal forms exist, but they are less commonly used in typical business applications.
- Normalization is performed **one table at a time**.
 - Identify the table's **functional dependencies**.
 - Check whether the table satisfies the rules of **1NF, 2NF, and 3NF**.
 - If necessary, divide the table into smaller, better-structured tables.

The Normalization Process (3 of 4)

Normal Forms

Normal Form	Characteristic	
First normal form (1NF)	No repeating groups; each field contains a single value; primary key identified	
Second normal form (2NF)	1NF and no partial dependencies	
Third normal form (3NF)	2NF and no transitive dependencies	
Boyce-Codd normal form (BCNF)	3NF and every determinant is a candidate key (special case of 3NF)	
Fourth normal form (4NF)	BCNF and no independent multivalued dependencies	
Fifth normal form (5NF or PJNF)	4NF and cannot have lossless decomposition into smaller tables	

The Normalization Process (4 of 4)

➤ Partial dependency

- Occurs when a non-key attribute depends on only **part of a composite primary key**.
- Partial dependencies are removed when moving to **2NF**.
- **Example:** (StudentID, CourseID) → StudentName
StudentName depends only on StudentID, not on the whole composite key.

➤ Transitive dependency

- Occurs when a non-key attribute depends on **another non-key attribute** instead of depending directly on the primary key.
- Transitive dependencies are removed when moving to **3NF**.
- **Example:**
EmployeeID → DepartmentID → DepartmentName
DepartmentName depends on DepartmentID, not directly on EmployeeID.

Knowledge Check Activity 6-1

- **What is normalization?**

Knowledge Check Activity : Answer

What is normalization?

Answer: Normalization is the process of organizing database tables to reduce unnecessary data redundancy and improve data integrity.

It helps:

- Prevent **insertion, update, and deletion anomalies**
- Reduce duplicated data
- Ensure attributes are stored in the appropriate tables

Conversion to First Normal Form (1NF) (1 of 3)

- **Repeating group:** A repeating group occurs when a single record contains multiple values for the same type of information.
- Repeating groups make the table harder to manage and can lead to redundancy.
 - In First **Normal Form (1NF)**, each field must contain one atomic value only.

Steps in Normalization (First Normal Form - 1NF):

1. Remove repeating groups
2. Identify the primary key
3. Identify all dependencies

1NF Checklist

- ✓ One value per field
- ✓ No repeating groups
- ✓ Primary key identified

Conversion to First Normal Form (1NF) (2 of 2)

Example: Remove a repeating group

Before 1NF

STUDENT_ID	STUDENT_NAME	COURSES
101	Maya	DB, Java
102	Noah	Networking



After 1NF

STUDENT_ID	STUDENT_NAME	COURSE
101	Maya	DB
101	Maya	Java
102	Noah	Networking

Problem: "DB, Java" stores two values in one field.

At this point, the table is in 1NF. Dependency analysis comes next when we prepare for 2NF and 3NF.

Conversion to Second Normal Form (2NF) (1 of 3)

➤ A table is in 2NF when **it is in 1NF and has no partial dependencies**.

- Partial dependencies can occur only when the primary key is composite.
- If a 1NF table has a single-attribute primary key, it is automatically in 2NF.

Dependency Diagram

A dependency diagram shows the functional dependencies among the attributes in a table.

We use it to:

- Identify partial dependencies before converting from 1NF to 2NF.
- Later, identify transitive dependencies before converting from 2NF to 3NF.

Conversion to Second Normal Form (2NF) (2 of 4)

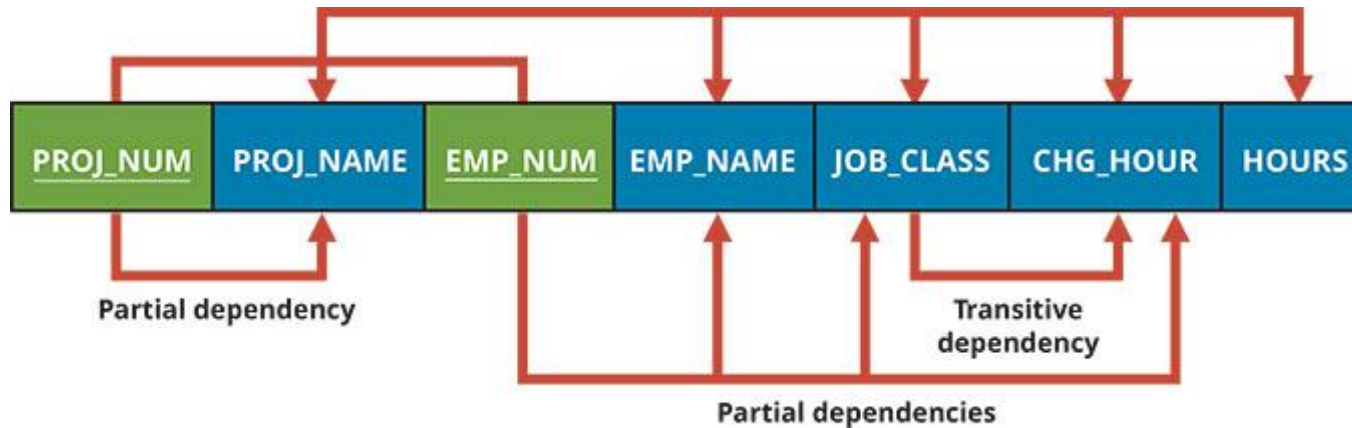
➤ **The 1NF-to-2NF conversion** is simple, you take the following steps:

- **Step 1:** Make new tables to eliminate partial dependencies
- **Step 2:** Reassign corresponding dependent attributes

➤ A table is in 2NF under the following circumstances :

- When it is in 1NF
- When it includes no partial dependencies

Conversion to Second Normal Form (2NF) (3 of 4)



1NF (PROJ_NUM, EMP_NUM, PROJ_NAME, EMP_NAME, JOB_CLASS, CHG_HOURS, HOURS)

PARTIAL DEPENDENCIES:

(PROJ_NUM $\Rightarrow$ PROJ_NAME)

(EMP_NUM $\Rightarrow$ EMP_NAME, JOB_CLASS, CHG_HOUR)

TRANSITIVE DEPENDENCY:

(JOB_CLASS $\Rightarrow$ CHG_HOUR)

What to identify

- Composite primary key: (PROJ_NUM, EMP_NUM)
- PROJ_NUM $\rightarrow$ PROJ_NAME
- EMP_NUM $\rightarrow$ EMP_NAME, JOB_CLASS, CHG_HOUR
- These are partial dependencies because they depend on only part of the composite key.

Goal for 2NF: remove the partial dependencies by creating separate tables.

Conversion to 2NF Results (4 of 4)

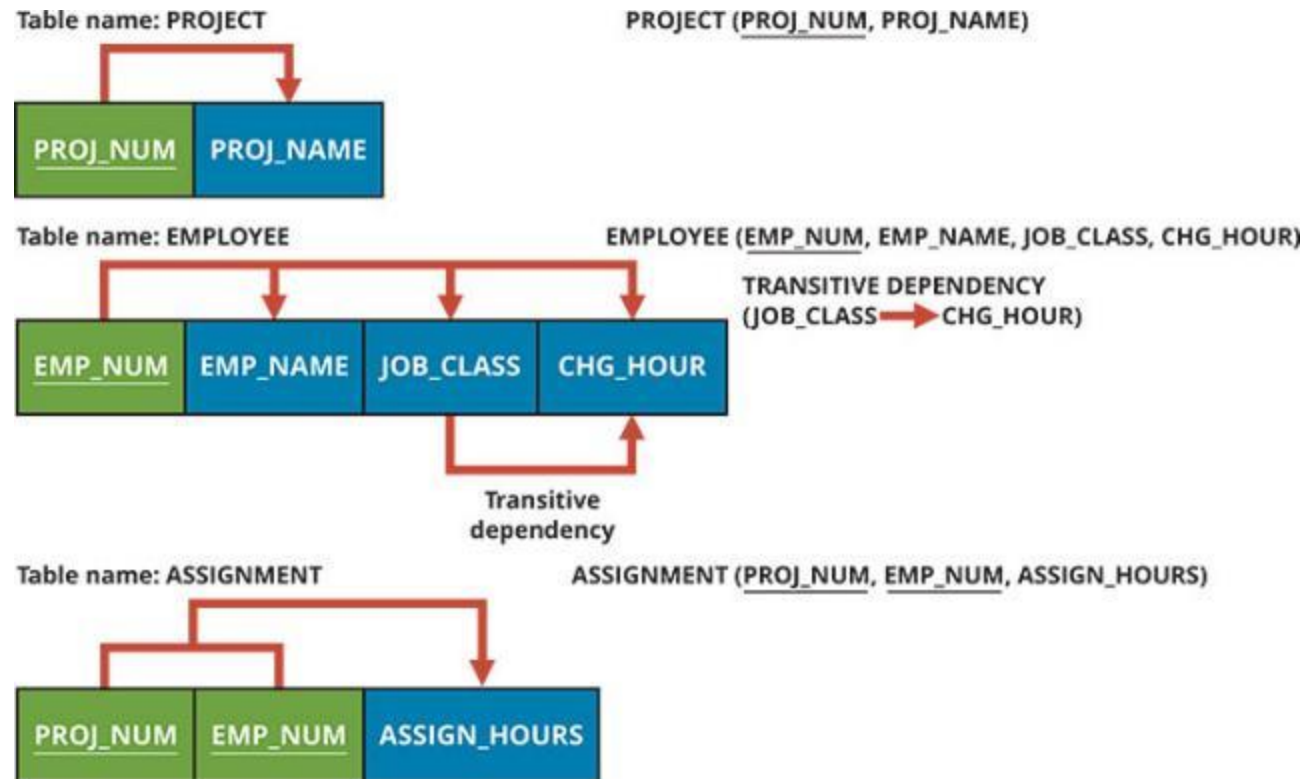


Figure 2: Second Normal Form (2NF) Conversion Results

Conversion to Third Normal Form (3NF) (1 of 2)

- A table is in 3NF when **it is in 2NF and has no transitive dependencies.**
- **Transitive dependency:** a non-key attribute depends on another non-key attribute.

Example: $\text{EMP_NUM} \rightarrow \text{JOB_CLASS} \rightarrow \text{CHG_HOUR}$
CHG_HOUR depends on JOB_CLASS, not directly on EMP_NUM.

Steps to Convert to 3NF

1. Create a new table for each transitive dependency.
2. Move the dependent attribute(s) to the new table and keep the determinant as a key/foreign key.

Conversion to Third Normal Form (3NF) (2 of 2)



Table name: PROJECT

PROJECT (PROJ_NUM, PROJ_NAME)



Table name: JOB

JOB (JOB_CLASS, CHG_HOUR)

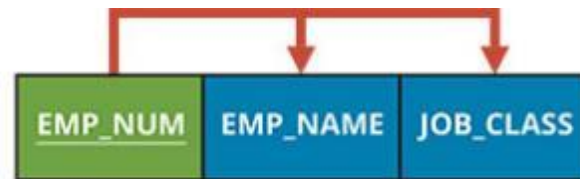


Table name: EMPLOYEE

EMPLOYEE (EMP_NUM, EMP_NAME, JOB_CLASS)



Table name: ASSIGNMENT

ASSIGNMENT (PROJ_NUM, EMP_NUM, ASSIGN_HOURS)

Figure 3 Third Normal Form (3NF)
Conversion Results

Knowledge Check Activity

- **When is a table in 3NF?**

Summary

Now that the lesson has ended, you should be able to:

1. Explain normalization and its role in the database design process
2. Identify each of the normal forms: 1NF, 2NF, 3NF
3. Explain how normal forms can be transformed from lower normal forms to higher normal forms
4. Apply normalization rules to evaluate and correct table structures

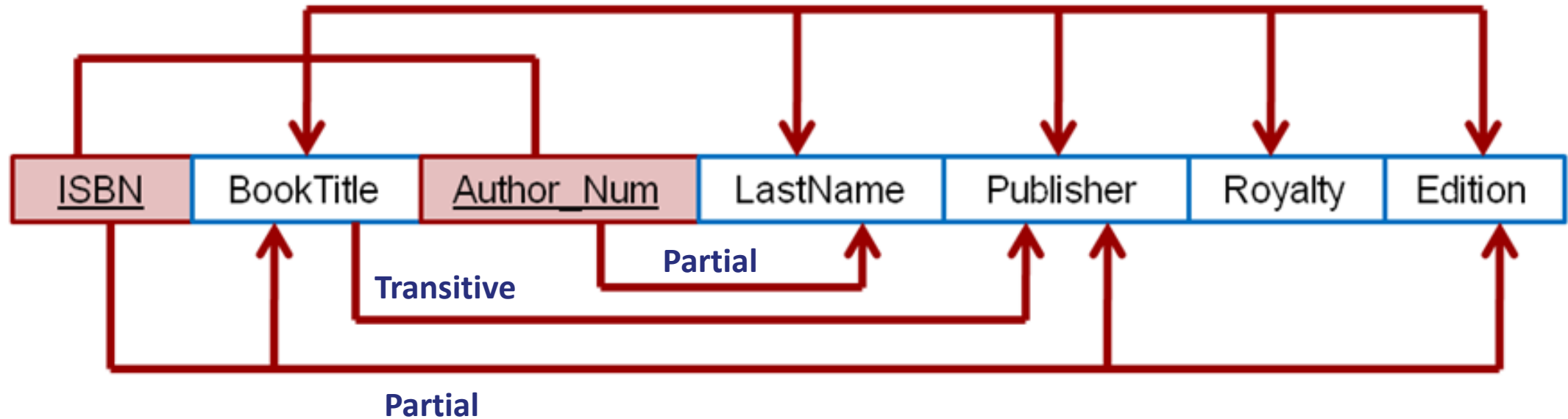


Class Activity: Book Royalty

Write the **relational schema**, draw its **dependency diagram** and **identify all dependencies**, including all partial and transitive dependencies and normalize the table to **2NF** and **3NF**.

<u>ISBN</u>	BookTitle	<u>Author_Num</u>	LastName	Publisher	Royalty	Edition
-------------	-----------	-------------------	----------	-----------	---------	---------

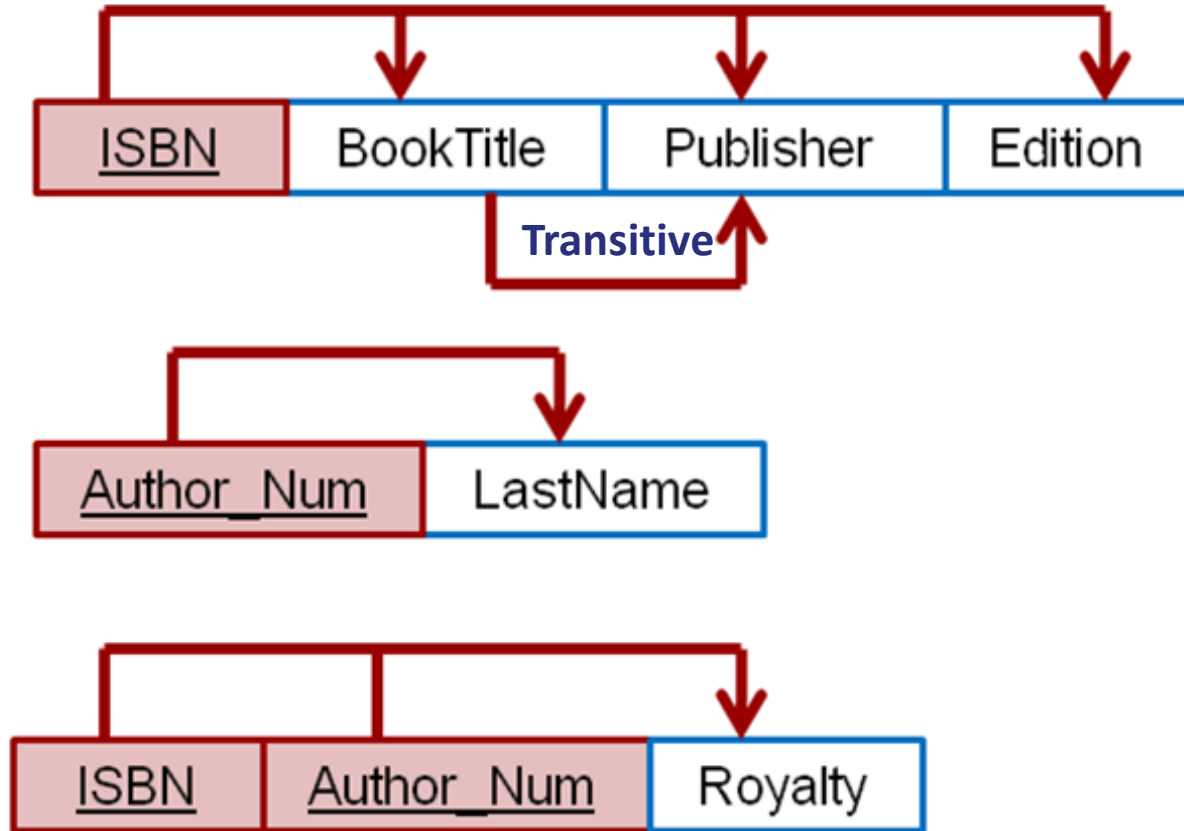
Book Royalty Dependency Diagram and relational schema



Relational Schema **1NF** (ISBN, Author_num, BookTitle, LastName, Publisher, Royalty, Edition)

2NF Normalization Results

Dependency Diagram



Relational Schema

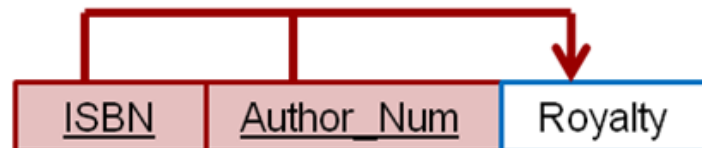
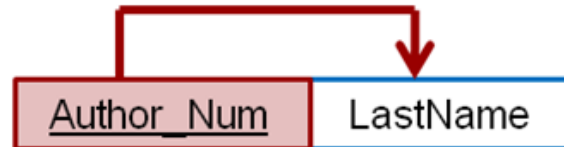
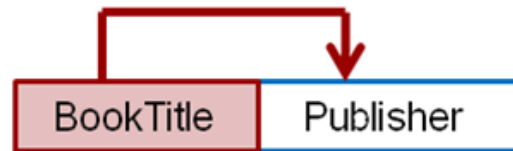
2NF (ISBN, BookTitle, Publisher, Edition)

3NF (Author_num, LastName)

3NF (ISBN, Authot_num, Royalty)

3NF Normalization Results

Dependency Diagram



Relational Schema

3NF (ISBN, BookTitle Edition)

3NF (BookTitle, Publisher)

3NF (Author_num, LastName)

3NF (ISBN, Authot_num, Royalty)